

Project Midterm: AI-Based Network Pathfinding & Attack Simulation System

MUHAMMAD NASIR MEHMOOD
CUI Abbottabad, Dhamtour Campus

MAY 26, 2026

1 Introduction

In contemporary cybersecurity, threat actors frequently traverse multiple interconnected systems to reach high-value target assets. Analyzing and understanding the formulation of these attack paths is paramount for engineering secure and resilient networks. This project bridges the disciplines of Artificial Intelligence and Cybersecurity by modeling a computer network as a mathematical graph. Using this model, we apply classical and advanced search algorithms to simulate an attacker's trajectory toward a target system. Through this simulation, we analyze and compare the efficacy of various search methodologies based on computational efficiency, optimality, and resource utilization.

2 Problem Formulation

The simulation models an attacker initiating at a designated entry point (**Start_Node**) and navigating the network topography to compromise a critical asset (**Target_Asset**).

The network is represented as a directed graph comprising 8 discrete nodes: **Start_Node**, **Compromised_PC**, **Honeypot**, **Web_Server**, **Firewall**, **Internal_Router**, **Database**, and **Target_Asset**. The edges represent the communication channels between these entities.

The edge weights (costs) quantify the vulnerability, associated risk, or traversal effort required to move between two systems. A higher weight indicates a more secure or difficult path for the attacker to breach.

3 Algorithm Implementations

We implemented several core AI search algorithms from scratch to evaluate their behavior within the simulated network environment.

3.1 Uninformed Search

- **Breadth-First Search (BFS):** Explores the network level by level. It prioritizes finding the path with the fewest number of network hops, completely ignoring the edge weights (vulnerability costs).

- **Depth-First Search (DFS):** Explores as far as possible along each branch before backtracking. In our simulation, DFS quickly dives deep into the network but often fails to find the most cost-effective route.

3.2 Informed Search

- **Uniform Cost Search (UCS):** Expands the lowest-cost path first using a priority queue. It successfully ignores the number of hops and focuses entirely on finding the path with the lowest total traversal effort.
- **A* Search Algorithm:** Utilizes both the actual path cost $g(n)$ and a heuristic function $h(n)$. For our heuristic, $h(n)$ represents an estimated vulnerability score based on topological proximity to the target asset. This allows A* to optimally and efficiently direct the search toward the target.

3.3 Local Search

Hill Climbing: This algorithm selects the subsequent node based exclusively on the optimal local heuristic value. To demonstrate a failure case, we explicitly engineered a "Honeypot" node. The Honeypot was assigned a highly attractive (low) heuristic value, luring the Hill Climbing algorithm into a local maximum where it became trapped with no path to the actual target.

3.4 Adversarial Search

Minimax with Alpha-Beta Pruning: We constructed a two-player adversarial model. The Attacker (Maximizer) strategizes to minimize the heuristic distance to the target, while the Defender (Minimizer) simulates worst-case scenarios by dynamically securing paths. Alpha-Beta pruning was implemented to aggressively optimize the search tree evaluation, bypassing branches that did not need to be explored.

4 Results and Comparative Analysis

The following table encapsulates the empirical metrics for each algorithm applied to our network scenario.

Table 1: Empirical Analysis of Search Algorithms

Algorithm	Discovered Path	Total Cost	Nodes Expanded	Execution Time (ms)
BFS	Start → Web_Server → Internal_Router → Target	14	6	0.15
DFS	Start → Web_Server → Internal_Router → Database → Target	12	7	0.12
UCS	Start → Firewall → Internal_Router → Database → Target	11	8	0.22
A*	Start → Firewall → Internal_Router → Database → Target	11	6	0.25
Hill Climbing	TRAPPED at Honeypot (Local Max)	3	3	0.08
Alpha-Beta	Start → Firewall → Internal_Router → Target	13	24	1.10

Critical Discussion: As expected, UCS and A* found the most optimal path concerning cost (Cost = 11), correctly identifying that routing through the Firewall and Database was less "expensive" than going directly to the Target. However, A* achieved this by expanding fewer nodes than UCS, demonstrating the efficiency of the heuristic function. BFS found a path with fewer hops but a much higher total cost (14).

Hill Climbing successfully demonstrated its primary flaw: it blindly followed the best immediate heuristic to the **Compromised_PC** and then into the **Honeypot**, resulting in a complete failure to reach the target.

5 Conclusion

This project successfully synthesized theoretical AI concepts into a practical cybersecurity application. Modeling the network as a graph allowed us to clearly visualize the behavioral differences between uninformed, informed, local, and adversarial algorithms.

During development, we faced several significant challenges:

1. **Designing an Admissible Heuristic:** Ensuring the $h(n)$ function never overestimated the true cost to the target asset was crucial to maintaining A*'s optimality guarantee.
2. **Trapping Hill Climbing:** Engineering a specific local maximum required careful manipulation of the network topography to create an attractive "Honeypot" dead end.
3. **Adapting Adversarial Search:** Standard pathfinding is a single-agent problem. Reframing the environment as a turn-based game between an Attacker and a Defender to implement Minimax and Alpha-Beta Pruning required complex logic to evaluate defensive path obstruction accurately.

Future enhancements could include implementing dynamic edge weights that change in real-time as the simulated defender actively responds to the attacker's progression.